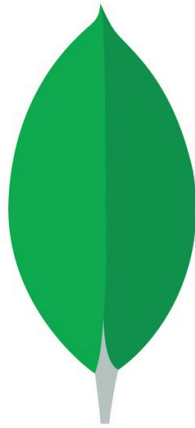


Actividad 1 UT 4 – MongoDB Atlas



mongoDB®

Índice

Explicación del código	3
Capturas de pantalla	14
Código completo	24
Acceso al zip del proyecto	32

Explicación del código

Main.java:

```
package com.example;

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        ClienteDAO clienteDAO = new ClienteDAO();
        VehiculoDAO vehiculoDAO = new VehiculoDAO();

        int opcion;

        do {
            System.out.println("\n--- MENÚ CONCESIONARIO ---");
            System.out.println("1. Insertar cliente");
            System.out.println("2. Listar clientes");
            System.out.println("3. Insertar vehículo");
            System.out.println("4. Listar todos los vehículos");
            System.out.println("5. Listar vehículos de un cliente");
            System.out.println("6. Marcar vehículo como vendido");
            System.out.println("7. Eliminar cliente");
            System.out.println("0. Salir");
            System.out.print("Opción: ");

            opcion = sc.nextInt();
            sc.nextLine();

            switch (opcion) {
                case 1:
                    System.out.print("DNI: ");
                    String dni = sc.nextLine();
                    System.out.print("Nombre: ");
                    String nombre = sc.nextLine();
                    System.out.print("Teléfono: ");
                    String telefono = sc.nextLine();
                    clienteDAO.insertarCliente(new Cliente(dni, nombre,
telefono));
```

```

        break;

    case 2:
        clienteDAO.listarClientes();
        break;

    case 3:
        System.out.print("Matrícula: ");
        String matricula = sc.nextLine();
        System.out.print("Marca: ");
        String marca = sc.nextLine();
        System.out.print("Modelo: ");
        String modelo = sc.nextLine();
        System.out.print("Precio: ");
        double precio = sc.nextDouble();
        sc.nextLine();
        System.out.print("DNI cliente: ");
        String dniCliente = sc.nextLine();
        vehiculoDAO.insertarVehiculo(
            new Vehiculo(matricula, marca, modelo,
precio, dniCliente));
        break;

    case 4:
        vehiculoDAO.listarVehiculos();
        break;

    case 5:
        System.out.print("DNI cliente: ");
        String dniBuscar = sc.nextLine();
        vehiculoDAO.listarVehiculosPorCliente(dniBuscar);
        break;

    case 6:
        System.out.print("Matrícula: ");
        String mat = sc.nextLine();
        vehiculoDAO.marcarVendido(mat);
        break;

    case 7:
        System.out.print("DNI cliente: ");
        String dniEliminar = sc.nextLine();
        clienteDAO.eliminarCliente(dniEliminar);

```

```
                break;
            }

        } while (opcion != 0);

        sc.close();
    }
}
```

La clase Main es el punto de entrada de la aplicación y contiene el método main, que se ejecuta al iniciar el programa. Su función principal es mostrar un menú por consola y gestionar la interacción del usuario con el sistema. Para ello, se utiliza un objeto Scanner que permite leer los datos introducidos desde el teclado.

Al comienzo del método main se crean las instancias de ClienteDAO y VehiculoDAO, que serán las encargadas de comunicarse con la base de datos MongoDB. Estas instancias permiten separar la lógica del acceso a datos del resto de la aplicación, siguiendo el patrón DAO.

El programa funciona mediante un bucle do-while que se repite hasta que el usuario selecciona la opción de salir. En cada iteración se muestra el menú del concesionario y se solicita una opción numérica. A continuación, un switch evalúa la opción elegida y ejecuta la funcionalidad correspondiente, como insertar clientes, listar clientes, insertar vehículos o marcar un vehículo como vendido.

Cada caso del menú solicita los datos necesarios al usuario, crea los objetos correspondientes y llama a los métodos adecuados de los DAO. De esta forma, la clase Main se limita a gestionar la entrada y salida de datos, dejando el trabajo con la base de datos a las clases especializadas. Al finalizar el programa, el Scanner se cierra correctamente.

Cliente.java:

```
package com.example;

public class Cliente {
    private String dni;
    private String nombre;
    private String telefono;

    public Cliente(String dni, String nombre, String telefono) {
```

```
        this.dni = dni;
        this.nombre = nombre;
        this.telefono = telefono;
    }

    public String getDni() {
        return dni;
    }

    public String getNombre() {
        return nombre;
    }

    public String getTelefono() {
        return telefono;
    }
}
```

La clase Cliente representa el modelo de un cliente del concesionario. Es una clase sencilla que define los atributos básicos que identifican a un cliente: el DNI, el nombre y el teléfono. Estos atributos coinciden con los campos que se almacenan en la colección clientes de MongoDB.

El constructor permite crear un objeto Cliente inicializando todos sus atributos. Además, la clase proporciona métodos getter para acceder a los valores de cada campo. No se incluyen métodos setter porque, en este caso, los datos del cliente no se modifican una vez creados, ya que el ejercicio no lo requiere.

Esta clase actúa como un puente entre la aplicación Java y los documentos almacenados en MongoDB, facilitando el trabajo con datos de forma orientada a objetos.

Vehiculo.java:

```
package com.example;

public class Vehiculo {
    private String matricula;
    private String marca;
    private String modelo;
    private double precio;
    private String dniCliente;
}
```

```
private boolean vendido;

    public Vehiculo(String matricula, String marca, String modelo,
double precio, String dniCliente) {
    this.matricula = matricula;
    this.marca = marca;
    this.modelo = modelo;
    this.precio = precio;
    this.dniCliente = dniCliente;
    this.vendido = false;
}

    public String getMatricula() {
        return matricula;
    }

    public String getMarca() {
        return marca;
    }

    public String getModelo() {
        return modelo;
    }

    public double getPrecio() {
        return precio;
    }

    public String getDniCliente() {
        return dniCliente;
    }

    public boolean isVendido() {
        return vendido;
    }
}
```

La clase Vehiculo define el modelo de un vehículo del concesionario. Contiene los atributos matrícula, marca, modelo, precio, el DNI del cliente asociado y un campo booleano que indica si el vehículo ha sido vendido. Estos atributos reflejan directamente la estructura de los documentos de la colección vehiculos.

El constructor recibe los datos principales del vehículo y asigna automáticamente el valor false al atributo vendido, ya que un vehículo recién insertado no se considera vendido por defecto. Al igual que en la clase Cliente, se incluyen únicamente métodos getter para acceder a los valores de los atributos.

Esta clase permite trabajar con los vehículos de forma clara y estructurada dentro de la aplicación, manteniendo la coherencia con los datos almacenados en MongoDB.

ClienteDAO.java:

```
package com.example;

import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import org.bson.Document;

import static com.mongodb.client.model.Filters.eq;

public class ClienteDAO {

    private MongoCollection<Document> clientes;
    private MongoCollection<Document> vehiculos;

    public ClienteDAO() {
        MongoDatabase db = MongoDBConnection.getDatabase();
        clientes = db.getCollection("clientes");
        vehiculos = db.getCollection("vehiculos");
    }

    public void insertarCliente(Cliente c) {
        Document doc = new Document("dni", c.getDni())
            .append("nombre", c.getNombre())
            .append("telefono", c.getTelefono());
        clientes.insertOne(doc);
    }

    public void listarClientes() {
        for (Document d : clientes.find()) {
            System.out.println(d.toJson());
        }
    }
}
```



```

public void buscarClientePorDni(String dni) {
    Document d = clientes.find(eq("dni", dni)).first();
    if (d != null) {
        System.out.println(d.toJson());
    }
}

public void eliminarCliente(String dni) {
    long count = vehiculos.countDocuments(eq("dniCliente", dni));
    if (count == 0) {
        clientes.deleteOne(eq("dni", dni));
        System.out.println("Cliente eliminado");
    } else {
        System.out.println("No se puede eliminar, tiene vehículos
asociados");
    }
}
}

```

La clase ClienteDAO se encarga de todas las operaciones relacionadas con la colección clientes de la base de datos. Además, también accede a la colección vehiculos para poder comprobar si un cliente tiene vehículos asociados antes de ser eliminado.

En el constructor se obtiene la conexión a la base de datos mediante la clase MongoDBConnection y se inicializan las colecciones clientes y vehiculos. Esto permite reutilizar la conexión en todos los métodos del DAO.

El método insertarCliente recibe un objeto Cliente, crea un Document de MongoDB con sus datos y lo inserta en la colección. El método listarClientes recorre todos los documentos de la colección y los muestra por consola en formato JSON.

El método buscarClientePorDni permite localizar un cliente concreto utilizando su DNI como filtro. Si el cliente existe, se muestra su información por pantalla. Por último, el método eliminarCliente comprueba primero si existen vehículos asociados a ese DNI en la colección vehiculos. Si no hay ninguno, el cliente se elimina; en caso contrario, se muestra un mensaje indicando que no es posible eliminarlo. De esta forma se gestiona manualmente la relación entre colecciones, ya que MongoDB no impone claves foráneas.

VehiculoDAO.java:

```

package com.example;

import org.bson.Document;

import com.mongodb.client.MongoCollection;
import static com.mongodb.client.model.Filters.eq;
import static com.mongodb.client.model.Updates.set;

public class VehiculoDAO {

    private MongoCollection<Document> vehiculos;

    public VehiculoDAO() {
        vehiculos = MongoDBConnection
            .getDatabase()
            .getCollection("vehiculos");
    }

    public void insertarVehiculo(Vehiculo v) {
        Document doc = new Document("matricula", v.getMatricula())
            .append("marca", v.getMarca())
            .append("modelo", v.getModelo())
            .append("precio", v.getPrecio())
            .append("dniCliente", v.getDniCliente())
            .append("vendido", false);
        vehiculos.insertOne(doc);
    }

    public void listarVehiculos() {
        for (Document d : vehiculos.find()) {
            System.out.println(d.toJson());
        }
    }

    public void listarVehiculosPorCliente(String dni) {
        vehiculos.find(eq("dniCliente", dni))
            .forEach(d -> System.out.println(d.toJson()));
    }

    public void marcarVendido(String matricula) {
        vehiculos.updateOne(eq("matricula", matricula),
            set("vendido", true));
    }
}

```

```
public void eliminarVehiculo(String matricula) {  
    vehiculos.deleteOne(eq("matricula", matricula));  
}  
}
```

La clase VehiculoDAO gestiona todas las operaciones relacionadas con la colección vehiculos. En el constructor se obtiene la colección correspondiente a partir de la conexión a MongoDB.

El método insertarVehiculo crea un documento con los datos del vehículo y lo inserta en la colección, estableciendo el campo vendido en false. El método listarVehiculos muestra todos los vehículos almacenados en la base de datos, recorriendo la colección completa.

El método listarVehiculosPorCliente permite filtrar los vehículos por el DNI del cliente, mostrando únicamente aquellos que pertenecen a una persona concreta. El método marcarVendido actualiza el campo vendido de un vehículo específico, identificado por su matrícula, cambiando su valor a true. Finalmente, el método eliminarVehiculo elimina un vehículo de la base de datos utilizando la matrícula como criterio de búsqueda.

MongoDBConnection.java:

```
package com.example;  
  
import com.mongodb.client.MongoClient;  
import com.mongodb.client.MongoClients;  
import com.mongodb.client.MongoDatabase;  
  
public class MongoDBConnection {  
  
    // Cadena de conexión a MongoDB Atlas  
    private static final String URI =  
"mongodb+srv://rmarmay2004:abdulmahamatamahaaa1234@cluster0.ygho6os.mon  
godb.net/?appName=Cluster0";  
  
    // Nombre de la base de datos  
    private static final String DATABASE = "concesionarioDB";  
  
    public static MongoDatabase getDatabase() {  
        MongoClient client = MongoClients.create(URI);  
        return client.getDatabase(DATABASE);  
    }  
}
```

```
}  
}
```

La clase `MongoDBConnection` centraliza la conexión con MongoDB Atlas. Contiene la cadena de conexión al clúster, que incluye el usuario, la contraseña y la dirección del servidor, así como el nombre de la base de datos que se va a utilizar.

El método `getDatabase` crea un objeto `MongoClient` a partir de la URI y devuelve la base de datos `concesionarioDB`. De esta forma, todas las clases DAO pueden acceder a la base de datos de manera uniforme, evitando duplicar código y facilitando el mantenimiento del proyecto.

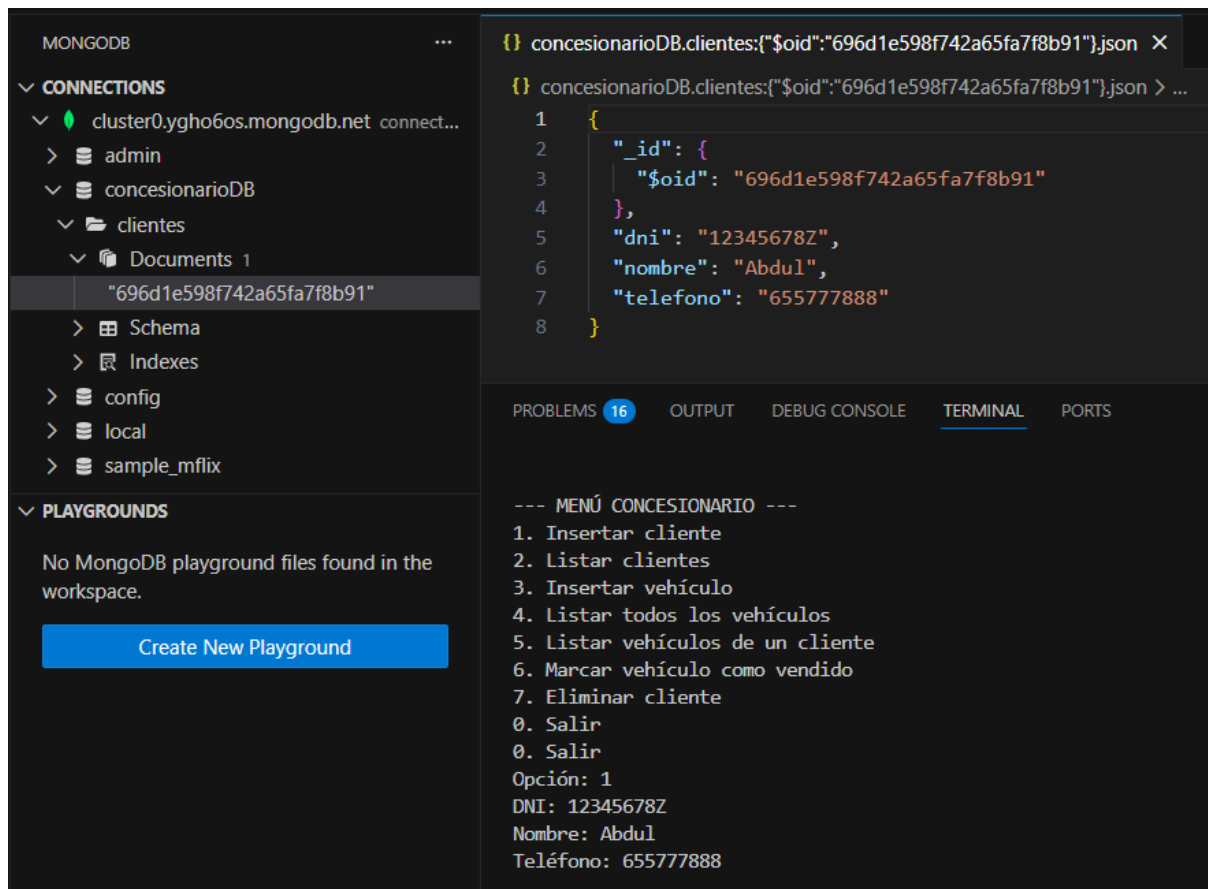
pom.xml:

```
<?xml version="1.0" encoding="UTF-8"?>  
<project xmlns="http://maven.apache.org/POM/4.0.0"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
http://maven.apache.org/xsd/maven-4.0.0.xsd">  
    <modelVersion>4.0.0</modelVersion>  
  
    <groupId>com.example</groupId>  
    <artifactId>demo</artifactId>  
    <version>1.0-SNAPSHOT</version>  
  
    <properties>  
        <maven.compiler.source>17</maven.compiler.source>  
        <maven.compiler.target>17</maven.compiler.target>  
    </properties>  
  
    <dependencies>  
        <dependency>  
            <groupId>org.mongodb</groupId>  
            <artifactId>mongodb-driver-sync</artifactId>  
            <version>4.11.1</version>  
        </dependency>  
    </dependencies>  
  
</project>
```

El fichero pom.xml define la configuración del proyecto Maven. En él se especifican los datos básicos del proyecto, como el groupId, el artifactId y la versión. También se indica que el proyecto utiliza Java 17 como versión de compilación.

En la sección de dependencias se incluye el driver oficial de MongoDB para Java, en su versión síncrona. Esta dependencia es imprescindible para poder conectar la aplicación Java con MongoDB Atlas y realizar operaciones sobre la base de datos. Gracias a Maven, esta librería se descarga y gestiona automáticamente, simplificando la configuración del proyecto.

Capturas de pantalla



MONGODB

...

CONNECTIONS

cluster0.ygho6os.mongodb.net connect...

admin

concesionarioDB

clientes

Documents 1

696d1e598f742a65fa7f8b91

Schema

Indexes

config

local

sample_mflix

PLAYGROUNDS

HELP AND FEEDBACK

What's New

Extension Documentation

MongoDB Documentation

Suggest a Feature

Report a Bug

Create Free Atlas Cluster

concesionarioDB.clientes:{"\$oid":"696d1e598f742a65fa7f8b91"}json

concesionarioDB.clientes:{"\$oid":"696d1e598f742a65fa7f8b91"}json

telefono

1 {

2 "_id": {

3 "\$oid": "696d1e598f742a65fa7f8b91"

4 },

5 "dni": "12345678Z",

6 "nombre": "Abdul",

7 "telefono": "655777888"

8 }

PROBLEMS 16

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Run

--- MENÚ CONCESIONARIO ---

1. Insertar cliente

2. Listar clientes

3. Insertar vehículo

4. Listar todos los vehículos

5. Listar vehículos de un cliente

6. Marcar vehículo como vendido

7. Eliminar cliente

0. Salir

Opción: 2

{"_id": {"\$oid": "696d1e598f742a65fa7f8b91"}, "dni": "12345678Z", "nombre": "Abdul", "telefono": "655777888"}

--- MENÚ CONCESIONARIO ---

1. Insertar cliente

2. Listar clientes

3. Insertar vehículo

4. Listar todos los vehículos

5. Listar vehículos de un cliente

6. Marcar vehículo como vendido

7. Eliminar cliente

0. Salir

Opción:

The screenshot displays the MongoDB Compass application. On the left, the 'CONNECTIONS' sidebar shows a cluster named 'cluster0.ygho6os.mongodb.net' with a database 'concesionarioDB' and a collection 'vehiculos'. The 'Documents' tab for 'vehiculos' is selected, showing a single document with the following fields:

```

{
  "_id": {
    "$oid": "696d23e70eef9c5e767be51d"
  },
  "matricula": "7777GGG",
  "marca": "Ford",
  "modelo": "Fiesta",
  "precio": 5000,
  "dniCliente": "12345678Z",
  "vendido": false
}

```

Below the document view, the 'TERMINAL' tab is active, showing a menu and the output of a command:

```

--- MENÚ CONCESIONARIO ---
1. Insertar cliente
2. Listar clientes
3. Insertar vehículo
4. Listar todos los vehículos
5. Listar vehículos de un cliente
6. Marcar vehículo como vendido
7. Eliminar cliente
0. Salir
Opción: 3
Matrícula: 7777GGG
Marca: Ford
Modelo: Fiesta
Precio: 5000
DNI cliente: 12345678Z

--- MENÚ CONCESIONARIO ---
1. Insertar cliente
2. Listar clientes
3. Insertar vehículo
4. Listar todos los vehículos
5. Listar vehículos de un cliente
6. Marcar vehículo como vendido
7. Eliminar cliente
0. Salir
Opción: 4
{"_id": {"$oid": "696d23e70eef9c5e767be51d"}, "matricula": "7777GGG", "marca": "Ford", "modelo": "Fiesta", "precio": 5000.0, "dniCliente": "12345678Z", "vendido": false}

```


The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a cluster named 'cluster0.ygho6os.mongodb.net' connected. The 'concesionarioDB' database is selected, and the 'vehiculos' collection is expanded, showing a single document with the ID '696d23e70eef9c5e767be51d'. The 'Terminal' tab is active, displaying the output of a Node.js application. The application has a menu with options: '1. Insertar cliente', '2. Listar clientes', '3. Insertar vehiculo', '4. Listar todos los vehiculos', '5. Listar vehiculos de un cliente', '6. Marcar vehiculo como vendido', '7. Eliminar cliente', and '0. Salir'. The user has selected '7. Eliminar cliente', and the application has successfully removed the document with the ID '696d23e70eef9c5e767be51d'.

The screenshot shows the VS Code interface with a MongoDB connection established. The left sidebar displays the 'CONNECTIONS' panel with a tree view showing the database structure: 'cluster0.ygho6os.mongodb.net' > 'concesionarioDB' > 'vehiculos' > 'Documents' > '696d23e70eef9c5e767be51d'. The main editor shows a JSON document in the 'concesionarioDB.vehiculos' collection:

```
{
  "_id": {
    "$oid": "696d23e70eef9c5e767be51d"
  },
  "matricula": "7777GGG",
  "marca": "Ford",
  "modelo": "Fiesta",
  "precio": 5000,
  "dniCliente": "12345678Z",
  "vendido": true
}
```

The bottom panel shows the 'TERMINAL' with the following output:

```
DNI cliente: 12345678Z
{"_id": {"$oid": "696d23e70eef9c5e767be51d"}, "matricula": "7777GGG", "marca": "Ford", "modelo": "Fiesta", "precio": 5000.0, "dniCliente": "12345678Z", "vendido": false}
678Z", "vendido": false}

--- MENÚ CONCESIONARIO ---
1. Insertar cliente
2. Listar clientes
3. Insertar vehículo
4. Listar todos los vehículos
5. Listar vehículos de un cliente
6. Marcar vehículo como vendido
7. Eliminar cliente
0. Salir
Opción: 6
Matricula: 7777GGG

--- MENÚ CONCESIONARIO ---
1. Insertar cliente
2. Listar clientes
3. Insertar vehículo
4. Listar todos los vehículos
5. Listar vehículos de un cliente
6. Marcar vehículo como vendido
7. Eliminar cliente
0. Salir
Opción: 4
{"_id": {"$oid": "696d23e70eef9c5e767be51d"}, "matricula": "7777GGG", "marca": "Ford", "modelo": "Fiesta", "precio": 5000.0, "dniCliente": "12345678Z", "vendido": true}

--- MENÚ CONCESIONARIO ---
1. Insertar cliente
```

MONGODB

CONNECTIONS

cluster0.ygho6os.mongodb.net connect...

admin

concesionarioDB

clientes

Documents 1

"696d1e598f742a65fa7f8b91"

Schema

PLAYGROUNDS

HELP AND FEEDBACK

What's New

Extension Documentation

MongoDB Documentation

Suggest a Feature

Report a Bug

Create Free Atlas Cluster

concesionarioDB.clientes:{"\$oid":"696d1e598f742a65fa7f8b91"}.json

concesionarioDB.clientes:{"\$oid":"696d1e598f742a65fa7f8b91"}.json > ...

1 {

2 "_id": {

3 "\$oid": "696d1e598f742a65fa7f8b91"

4 },

5 "dni": "12345678Z",

6 "nombre": "Abdul",

7 "telefono": "655777888"

8 }

PROBLEMS 16

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

0. Salir

Opción: 4

{ "_id": {"\$oid": "696d23e70eef9c5e767be51d"}, "matricula":

"precio": 5000.0, "dniCliente": "12345678Z", "vendido": tru

MENÚ CONCESIONARIO ---

1. Insertar cliente

2. Listar clientes

3. Insertar vehículo

4. Listar todos los vehículos

5. Listar vehículos de un cliente

6. Marcar vehículo como vendido

7. Eliminar cliente

0. Salir

Opción: 7

19

MONGODB

CONNECTIONS

cluster0.ygho6os.mongodb.net connect...

admin

concesionarioDB

clientes

Documents 1

"696d1e598f742a65fa7f8b91"

Schema

PLAYGROUNDS

HELP AND FEEDBACK

What's New

Extension Documentation

MongoDB Documentation

Suggest a Feature

Report a Bug

Create Free Atlas Cluster

concesionarioDB.clientes:{"_id":"696d1e598f742a65fa7f8b91"}.json

concesionarioDB.clientes:{"_id":"696d1e598f742a65fa7f8b91"}.json

1 {

2 "_id": {

3 "\$oid": "696d1e598f742a65fa7f8b91"

4 },

5 "dni": "12345678Z",

6 "nombre": "Abdul",

7 "telefono": "655777888"

8 } }

PROBLEMS 16

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

--- MENÚ CONCESIONARIO ---

1. Insertar cliente

2. Listar clientes

3. Insertar vehículo

4. Listar todos los vehículos

5. Listar vehículos de un cliente

6. Marcar vehículo como vendido

7. Eliminar cliente

0. Salir

Opción: 7

DNI cliente: 12345678Z

No se puede eliminar, tiene vehículos asociados

--- MENÚ CONCESIONARIO ---

1. Insertar cliente

2. Listar clientes

20

MONGODB

CONNECTIONS

cluster0.ygho6os.mongodb.net connect...

admin

concesionarioDB

clientes

Documents 2

"696d1e598f742a65fa7f8b91"

"696d27bf9cf0f53045bdbf2b"

PLAYGROUNDS

HELP AND FEEDBACK

What's New

Extension Documentation

MongoDB Documentation

Suggest a Feature

Report a Bug

Create Free Atlas Cluster

concesionarioDB.clientes:{"\$oid":"696d27bf9cf0f53045bdbf2b"}.json

concesionarioDB.clientes:{"\$oid":"696d27bf9cf0f53045bdbf2b"}.json

1

2

3

4

5

6

7

8

"_id": {

"\$oid": "696d27bf9cf0f53045bdbf2b"

}

,

"dni": "87654321A",

"nombre": "Walid",

"telefono": "777888999"

PROBLEMS 16

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

DNI cliente: 12345678Z

No se puede eliminar, tiene vehículos asociados

MENÚ CONCESIONARIO ---

1. Insertar cliente

2. Listar clientes

3. Insertar vehículo

4. Listar todos los vehículos

5. Listar vehículos de un cliente

6. Marcar vehículo como vendido

7. Eliminar cliente

0. Salir

Opción: 1

DNI: 87654321A

Nombre: Walid

Teléfono: 777888999

21

MONGODB

CONNECTIONS

cluster0.ygho6os.mongodb.net connect...

admin

concesionarioDB

clientes

Documents 1

"696d1e598f742a65fa7f8b91"

Schema

PLAYGROUNDS

HELP AND FEEDBACK

What's New

Extension Documentation

MongoDB Documentation

Suggest a Feature

Report a Bug

Create Free Atlas Cluster

PROBLEMS 16

OUTPUT

DEBUG CONSOLE

TERMINAL

PO

4. Listar todos los vehículos

5. Listar vehículos de un cliente

6. Marcar vehículo como vendido

7. Eliminar cliente

0. Salir

Opción: 1

DNI: 87654321A

Nombre: Walid

Teléfono: 777888999

--- MENÚ CONCESIONARIO ---

1. Insertar cliente

2. Listar clientes

3. Insertar vehículo

4. Listar todos los vehículos

5. Listar vehículos de un cliente

6. Marcar vehículo como vendido

7. Eliminar cliente

0. Salir

Opción: 7

DNI cliente: 87654321A

Cliente eliminado

--- MENÚ CONCESIONARIO ---

1. Insertar cliente

2. Listar clientes

22

```
6. Marcar vehículo como vendido
7. Eliminar cliente
0. Salir
Opción: 7
DNI cliente: 87654321A
Cliente eliminado

--- MENÚ CONCESIONARIO ---
1. Insertar cliente
2. Listar clientes
3. Insertar vehículo
4. Listar todos los vehículos
5. Listar vehículos de un cliente
6. Marcar vehículo como vendido
7. Eliminar cliente
0. Salir
Opción: 0
PS C:\Users\Asus\Desktop\act1ut4>
```

Código completo

Main.java:

```
package com.example;

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        ClienteDAO clienteDAO = new ClienteDAO();
        VehiculoDAO vehiculoDAO = new VehiculoDAO();

        int opcion;

        do {
            System.out.println("\n--- MENÚ CONCESIONARIO ---");
            System.out.println("1. Insertar cliente");
            System.out.println("2. Listar clientes");
            System.out.println("3. Insertar vehículo");
            System.out.println("4. Listar todos los vehículos");
            System.out.println("5. Listar vehículos de un cliente");
            System.out.println("6. Marcar vehículo como vendido");
            System.out.println("7. Eliminar cliente");
            System.out.println("0. Salir");
            System.out.print("Opción: ");

            opcion = sc.nextInt();
            sc.nextLine();

            switch (opcion) {
                case 1:
                    System.out.print("DNI: ");
                    String dni = sc.nextLine();
                    System.out.print("Nombre: ");
                    String nombre = sc.nextLine();
                    System.out.print("Teléfono: ");
                    String telefono = sc.nextLine();
                    clienteDAO.insertarCliente(new Cliente(dni, nombre,
telefono));
```



```

        break;

    case 2:
        clienteDAO.listarClientes();
        break;

    case 3:
        System.out.print("Matrícula: ");
        String matricula = sc.nextLine();
        System.out.print("Marca: ");
        String marca = sc.nextLine();
        System.out.print("Modelo: ");
        String modelo = sc.nextLine();
        System.out.print("Precio: ");
        double precio = sc.nextDouble();
        sc.nextLine();
        System.out.print("DNI cliente: ");
        String dniCliente = sc.nextLine();
        vehiculoDAO.insertarVehiculo(
            new Vehiculo(matricula, marca, modelo,
precio, dniCliente));
        break;

    case 4:
        vehiculoDAO.listarVehiculos();
        break;

    case 5:
        System.out.print("DNI cliente: ");
        String dniBuscar = sc.nextLine();
        vehiculoDAO.listarVehiculosPorCliente(dniBuscar);
        break;

    case 6:
        System.out.print("Matrícula: ");
        String mat = sc.nextLine();
        vehiculoDAO.marcarVendido(mat);
        break;

    case 7:
        System.out.print("DNI cliente: ");
        String dniEliminar = sc.nextLine();
        clienteDAO.eliminarCliente(dniEliminar);

```

```
                break;
            }

        } while (opcion != 0);

        sc.close();
    }
}
```

Cliente.java:

```
package com.example;

public class Cliente {
    private String dni;
    private String nombre;
    private String telefono;

    public Cliente(String dni, String nombre, String telefono) {
        this.dni = dni;
        this.nombre = nombre;
        this.telefono = telefono;
    }

    public String getDni() {
        return dni;
    }

    public String getNombre() {
        return nombre;
    }

    public String getTelefono() {
        return telefono;
    }
}
```

Vehiculo.java:

```
package com.example;

public class Vehiculo {
    private String matricula;
    private String marca;
    private String modelo;
    private double precio;
    private String dniCliente;
    private boolean vendido;

    public Vehiculo(String matricula, String marca, String modelo,
double precio, String dniCliente) {
        this.matricula = matricula;
        this.marca = marca;
        this.modelo = modelo;
        this.precio = precio;
        this.dniCliente = dniCliente;
        this.vendido = false;
    }

    public String getMatricula() {
        return matricula;
    }

    public String getMarca() {
        return marca;
    }

    public String getModelo() {
        return modelo;
    }

    public double getPrecio() {
        return precio;
    }

    public String getDniCliente() {
        return dniCliente;
    }

    public boolean isVendido() {
        return vendido;
    }
}
```

}

ClienteDAO.java:

```
package com.example;

import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import org.bson.Document;

import static com.mongodb.client.model.Filters.eq;

public class ClienteDAO {

    private MongoCollection<Document> clientes;
    private MongoCollection<Document> vehiculos;

    public ClienteDAO() {
        MongoDatabase db = MongoDBConnection.getDatabase();
        clientes = db.getCollection("clientes");
        vehiculos = db.getCollection("vehiculos");
    }

    public void insertarCliente(Cliente c) {
        Document doc = new Document("dni", c.getDni())
            .append("nombre", c.getNombre())
            .append("telefono", c.getTelefono());
        clientes.insertOne(doc);
    }

    public void listarClientes() {
        for (Document d : clientes.find()) {
            System.out.println(d.toJson());
        }
    }

    public void buscarClientePorDni(String dni) {
        Document d = clientes.find(eq("dni", dni)).first();
        if (d != null) {
            System.out.println(d.toJson());
        }
    }
}
```

```

    }

    public void eliminarCliente(String dni) {
        long count = vehiculos.countDocuments(eq("dniCliente", dni));
        if (count == 0) {
            clientes.deleteOne(eq("dni", dni));
            System.out.println("Cliente eliminado");
        } else {
            System.out.println("No se puede eliminar, tiene vehículos asociados");
        }
    }
}

```

VehiculoDAO.java:

```

package com.example;

import org.bson.Document;

import com.mongodb.client.MongoCollection;
import static com.mongodb.client.model.Filters.eq;
import static com.mongodb.client.model.Updates.set;

public class VehiculoDAO {

    private MongoCollection<Document> vehiculos;

    public VehiculoDAO() {
        vehiculos = MongoDBConnection
            .getDatabase()
            .getCollection("vehiculos");
    }

    public void insertarVehiculo(Vehiculo v) {
        Document doc = new Document("matricula", v.getMatricula())
            .append("marca", v.getMarca())
            .append("modelo", v.getModelo())
            .append("precio", v.getPrecio())
            .append("dniCliente", v.getDniCliente())
            .append("vendido", false);
    }
}

```

```

        vehiculos.insertOne(doc);
    }

    public void listarVehiculos() {
        for (Document d : vehiculos.find()) {
            System.out.println(d.toJson());
        }
    }

    public void listarVehiculosPorCliente(String dni) {
        vehiculos.find(eq("dniCliente", dni))
            .forEach(d -> System.out.println(d.toJson()));
    }

    public void marcarVendido(String matricula) {
        vehiculos.updateOne(eq("matricula", matricula),
            set("vendido", true));
    }

    public void eliminarVehiculo(String matricula) {
        vehiculos.deleteOne(eq("matricula", matricula));
    }
}

```

MongoDBConnection.java:

```

package com.example;

import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoDatabase;

public class MongoDBConnection {

    // Cadena de conexión a MongoDB Atlas
    private static final String URI =
"mongodb+srv://rmarmay2004:abdulmahamatamahaaa1234@cluster0.ygho6os.mon
godb.net/?appName=Cluster0";

    // Nombre de la base de datos
    private static final String DATABASE = "concesionarioDB";
}

```

```

public static MongoDBDatabase getDatabase() {
    MongoClient client = MongoClientUtils.create(URI);
    return client.getDatabase(DATABASE);
}
}

```

pom.xml:

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>demo</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.mongodb</groupId>
            <artifactId>mongodb-driver-sync</artifactId>
            <version>4.11.1</version>
        </dependency>
    </dependencies>

</project>

```

Acceso al zip del proyecto

<https://github.com/RafaelMayor/AED/tree/main/act1ut4>